

Rapport E5

Deploiement d'une application web sur Kubernetes avec Flux GitOps

BTS SIO -- Option SISR

Abidi Mohamed

Aurlom BTS+

Alternance -- Faculte de Pharmacie de Paris (Universite Paris-Saclay)

Annee 2025-2026

Sommaire

1. Contexte
 2. Objectifs
 3. Cahier des charges
 - 3.1. Contraintes techniques
 - 3.2. Contraintes de securite
 - 3.3. Flux de deploiement (GitOps)
 4. Solution et Architecture
 - 4.1. Description de la solution
 - 4.2. Schema d'architecture
 5. Prerequis et Environnement
 - 5.1. Environnement materiel et logiciel
 - 5.2. Composants de l'application
 6. Realisation et Configuration
 - 6.1. Creation des manifestes Kubernetes
 - 6.2. Configuration des variables d'environnement et secrets
 - 6.3. Construction et publication des images Docker
 - 6.4. Mise en place du pipeline CI/CD
 - 6.5. Resolution des incidents de deploiement
 7. Tests et Validation
 - 7.1. Verification des pods et services
 - 7.2. Test d'accès à l'application
 - 7.3. Creation des comptes utilisateurs
 8. Conclusion
-

1. Contexte

Dans le cadre de mon alternance au sein du service informatique de la Faculte de Pharmacie de Paris (Universite Paris-Saclay), j'interviens sur une infrastructure Kubernetes geree via l'approche GitOps avec Flux CD. Le cluster heberge plusieurs applications internes de la faculte (gestion reseau, examens, stages, assidueite), deployees de maniere declarative a partir d'un depot Git centralise.

La faculte a developpe une nouvelle application web interne : l'**Annuaire du Personnel**. Cette application permet de consulter et rechercher les membres du personnel de la Faculte de Pharmacie (enseignants-

chercheurs, personnel administratif, personnel technique). Elle se compose d'un backend API (Python/FastAPI), d'un frontend web (React/TypeScript) et de services annexes (PostgreSQL, MeiliSearch).

Ma mission consiste a configurer l'ensemble des manifestes Kubernetes necessaires au deploiement de cette application sur le cluster de production, a mettre en place le pipeline CI/CD pour automatiser les futures mises a jour, et a resoudre les incidents rencontres lors du premier deploiement.

2. Objectifs

Les objectifs de cette mission sont les suivants :

- Deployer l'application Annuaire sur le cluster Kubernetes de production en respectant les conventions etablies pour les autres applications de la faculte.
- Configurer l'ensemble des ressources Kubernetes : Deployments, Services, Ingress, ConfigMaps, Secrets, PersistentVolumeClaims.
- Mettre en place le pipeline CI/CD GitLab pour automatiser la construction des images Docker et le deploiement via Flux.
- Rendre l'application accessible sur le domaine `annuaire.pharmacie.u-pariscite.fr` avec un certificat TLS valide.
- Creer les comptes utilisateurs administrateurs pour la gestion de l'application.

3. Cahier des charges

3.1. Contraintes techniques

- L'application se compose de **quatre services** : un frontend (nginx), un backend (FastAPI/Python), une base de donnees PostgreSQL et un moteur de recherche MeiliSearch.
- Chaque service doit s'executer dans un pod dedie avec des limites de ressources CPU et memoire definies.
- Les donnees de PostgreSQL et MeiliSearch doivent etre persistees sur des volumes de type `iscsi-retain` (stockage Synology).
- Les images Docker sont hebergees sur le registry prive GitLab de la faculte (`gitlab.pharmacie.univ-paris5.fr:8443`), protege par un certificat auto-signe.
- Le deploiement doit suivre le meme modele que les autres applications existantes (assidueite, pharmexam) pour garantir l'homogeneite de l'infrastructure.

3.2. Contraintes de securite

- Les pods doivent s'executer en tant qu'utilisateur non-root (`runAsNonRoot: true`) avec un `securityContext` restreignant l'escalade de privileges.
- Les secrets (mots de passe PostgreSQL, cle MeiliSearch, credentials LDAP) sont stockes dans des objets Kubernetes `Secret` de type `Opaque`.
- L'acces au registry prive GitLab est assure par un secret `imagePullSecrets` de type `dockerconfigjson`, replique automatiquement dans chaque namespace par le controleur Reflector.
- La cle MeiliSearch doit faire au minimum 16 octets en environnement de production.
- L'acces a l'application est securise par HTTPS via un certificat Let's Encrypt genere automatiquement par cert-manager.

3.3. Flux de deploiement (GitOps)

Le deploiement suit le modele GitOps avec Flux CD :

1. Le developpeur pousse son code sur le depot GitLab de l'application (`pharmacie/app/annuaire`).
2. Il cree un tag Git (ex : `v0.0.1`).
3. Le pipeline GitLab CI construit l'image Docker via BuildKit et la pousse sur le registry.

- 4. Le job `deploy_flux_iac` met automatiquement a jour le tag de l'image dans le depot d'infrastructure (`pharmacie/infra/k8s-iac-flux`).
- 5. Flux detecte le changement dans le depot d'infrastructure (polling toutes les 5 minutes) et applique les nouveaux manifestes sur le cluster.
- 6. Kubernetes deploie la nouvelle version de l'application.

4. Solution et Architecture

4.1. Description de la solution

L'application Annuaire est deployee dans un namespace Kubernetes dedie (`annuaire`) avec quatre Deployments distincts :

Composant	Image	Port	Role
Frontend	<code>annuaire/frontend:v0.0.1</code>	8080	Interface web (nginx + fichiers statiques React)
Backend	<code>annuaire:v0.0.1</code>	8000	API REST FastAPI (Python 3.12)
PostgreSQL	<code>postgres:15-alpine</code>	5432	Base de donnees relationnelle
MeiliSearch	<code>getmeili/meilisearch:v1.6</code>	7700	Moteur de recherche full-text

Le routage est assure par un Ingress Traefik qui dirige les requetes :

- `/api` , `/docs` , `/redoc` , `/openapi.json` vers le backend (port 8000)
- `/` vers le frontend (port 8080)

Le backend utilise des **init containers** pour :

1. Attendre que PostgreSQL soit disponible (`wait-postgres`)
2. Attendre que MeiliSearch soit disponible (`wait-meilisearch`)
3. Importer les donnees initiales du personnel (`seed-annuaire`)

4.2. Schema d'architecture



[Pod PostgreSQL]	[Pod MeiliSearch]
postgres:15-alpine	meilisearch:v1.6
[PVC 20Gi]	[PVC 20Gi]
iscsi-retain	iscsi-retain

5. Prerequis et Environnement

5.1. Environnement materiel et logiciel

Element	Detail
Cluster Kubernetes	Multi-noeuds (k-node-004 a k-node-006)
Orchestrateur GitOps	Flux CD v2
Registry Docker	GitLab Registry (gitlab.pharmacie.univ-paris5.fr:8443)
CI/CD	GitLab CI avec template partage (pharmacie/ci-templates)
Reverse proxy	Traefik (ingress public, IP 194.254.93.5)
Certificats TLS	cert-manager + Let's Encrypt
Stockage persistant	Synology CSI, StorageClass iscsi-retain (RWO)
Depot infrastructure	pharmacie/infra/k8s-iac-flux
Depot application	pharmacie/app/annuaire

5.2. Composants de l'application

Composant	Technologie	Version
Backend API	Python / FastAPI / SQLAlchemy async	3.12
Frontend	React / TypeScript / Vite	Node 20
Base de donnees	PostgreSQL	15-alpine
Moteur de recherche	MeiliSearch	v1.6
Serveur web frontend	nginx-unprivileged	1.27-alpine
Authentification	Systeme interne (email/password, sessions cookie)	

6. Realisation et Configuration

6.1. Creation des manifestes Kubernetes

Le deploiement de l'application necessite la creation de sept fichiers de manifestes dans le repertoire `applications/annuaire/` du depot d'infrastructure :

Fichier	Ressource	Role
<code>namespace.yml</code>	Namespace	Cree le namespace annuaire avec les labels de la faculte

deploy.yaml	Deployment (x4)	Definit les 4 deployments (frontend, backend, postgres, meilisearch)
svc.yaml	Service (x4)	Expose les 4 services en ClusterIP
ingress.yaml	Ingress	Configure le routage Traefik avec TLS Let's Encrypt
config.yaml	ConfigMap (x2)	Variables d'environnement non sensibles
secret.yaml	Secret	Credentials PostgreSQL, MeiliSearch, LDAP
pvc.yaml	PersistentVolumeClaim (x2)	Volumes 20 Gi pour PostgreSQL et MeiliSearch

Chaque Deployment est annoté avec `reloader.stakater.com/auto: "true"` pour permettre le redémarrage automatique des pods lors d'un changement de ConfigMap ou de Secret.

6.2. Configuration des variables d'environnement et secrets

Le backend nécessite de nombreuses variables d'environnement pour fonctionner. Celles-ci sont réparties entre le ConfigMap (valeurs non sensibles) et le Secret (valeurs sensibles).

ConfigMap `annuaire-backend-config` :

Variable	Valeur	Description
DB_HOST	annuaire-postgres	Nom du service PostgreSQL
DB_PORT	5432	Port PostgreSQL
MEILISEARCH_URL	http://annuaire-meilisearch:7700	URL interne MeiliSearch
PORT	8000	Port d'écoute du backend
LDAP_URL	ldap://ldap.universite-paris-cite.fr	Serveur LDAP de l'université
LDAP_BASE_DN	dc=u-paris,dc=fr	Base DN LDAP
CORS_ORIGINS	https://annuaire.pharmacie.u-pariscite.fr	Origines CORS autorisées
SMTP_HOST	postfix.prod.svc	Serveur SMTP interne
SMTP_PORT	25	Port SMTP
SMTP_FROM	annuaire@u-paris.fr	Adresse expéditeur
ADMIN_EMAIL	tice.pharma@u-paris.fr	Email administrateur
BASE_URL	https://annuaire.pharmacie.u-pariscite.fr	URL publique
CAS_SERVER_URL	https://auth.u-paris.fr/idp/profile/cas	Serveur CAS université
CAS_SERVICE_URL	https://annuaire.pharmacie.u-pariscite.fr	URL de callback CAS

Les valeurs LDAP et CAS ont été déterminées en analysant les configurations des autres applications déployées (appscanetu, traineeship) et le fichier `.env.example` du projet.

Secret `annuaire-secrets` : contient les credentials PostgreSQL (`postgres-user` , `postgres-password` , `postgres-db`), la clé MeiliSearch (`meilisearch-master-key`) et les credentials LDAP (`ldap-bind-dn` , `ldap-bind-password`).

Correction PostgreSQL PGDATA : la StorageClass `iscsi-retain` cree des volumes au format bloc qui contiennent un repertoire `lost+found` . PostgreSQL refuse de s'initialiser dans un repertoire non vide. La solution consiste a ajouter la variable d'environnement `PGDATA` pointant vers un sous-repertoire :

```
- name: PGDATA
  value: /var/lib/postgresql/data/pgdata
```

Correction MeiliSearch : en environnement de production, MeiliSearch exige une cle maitre d'au minimum 16 octets. La cle initiale `masterKey` (9 octets) a ete remplacee par une cle securisee de 44 caracteres generee par MeiliSearch lui-meme.

6.3. Construction et publication des images Docker

Le registry GitLab utilise un certificat auto-signe (CA interne `RootCAPharma`). Pour pouvoir pousser des images depuis le poste de travail, il a fallu :

1. Extraire le certificat CA depuis le fichier `infra/cert-manager/ca-bundle.yaml` du depot d'infrastructure.
2. Copier ce certificat dans `/etc/docker/certs.d/gitlab.pharmacie.univ-paris5.fr:8443/ca.crt` .
3. Se connecter au registry : `docker login gitlab.pharmacie.univ-paris5.fr:8443` .

Les images ont ensuite ete construites et poussees manuellement pour le premier deploiement :

```
# Backend (depuis la racine du projet)
docker build -t gitlab.../pharmacie/app/annuaire:v0.0.1 .
docker push gitlab.../pharmacie/app/annuaire:v0.0.1

# Frontend
docker build -t gitlab.../pharmacie/app/annuaire/frontend:v0.0.1 ./frontend
docker push gitlab.../pharmacie/app/annuaire/frontend:v0.0.1
```

6.4. Mise en place du pipeline CI/CD

Pour automatiser les futurs deploiements, un pipeline GitLab CI a ete configure. Le CI de l'annuaire suit le meme modele que les autres applications (assidueite, pharmexam) :

```
include:
- project: pharmacie/ci-templates
  ref: main
  file: pipelines/container/container.yml

deploy_flux_iac:
  variables:
    AUTO_UPDATE_DIR: "applications/annuaire"

dockerfile_lint:
  allow_failure: true
```

Le template partage `pharmacie/ci-templates` gere automatiquement :

- La construction de l'image Docker via BuildKit (sur les tags Git uniquement).
- La publication sur le registry GitLab.

- La mise a jour automatique du tag d'image dans le depot `k8s-iac-flux` via le job `deploy_flux_iac`.

Ce template attend un **Dockerfile unique a la racine du projet**. Un Dockerfile racine a donc ete cree, reprenant le contenu du `backend/Dockerfile` avec les chemins adaptes (`COPY backend/ /app/` au lieu de `COPY . /app`).

Le workflow de deploiement est desormais :

```
git commit -m "feat: nouvelle fonctionnalite"
git push origin main
git tag v0.1.0
git push origin v0.1.0
# Le pipeline construit l'image, met a jour k8s-iac-flux, Flux deploie automatiquement
```

6.5. Resolution des incidents de deploiement

Plusieurs incidents ont ete rencontres et resolus lors du premier deploiement :

Incident	Cause	Resolution
ImagePullBackOff (frontend)	Pas de imagePullSecrets dans le Deployment frontend	Ajout de imagePullSecrets: [{name: gitlab-credentials}]
ImagePullBackOff (403 Forbidden)	Chemin d'image incorrect (annuaire-frontend au lieu de annuaire/frontend)	Correction du chemin vers pharmacie/app/annuaire/frontend
ImagePullBackOff (not found)	Tag 0.0.1 inexistant (le tag Git etait v0.0.1)	Ajout du prefixe v dans les tags d'image
CrashLoopBackOff (frontend)	nginx.conf contient un proxy_pass vers backend:8000 (nom inexistant en k8s)	Suppression du bloc location /api/ (l'Ingress gere le routage)
CrashLoopBackOff (MeiliSearch)	Cle maitre trop courte (9 octets, minimum 16 requis)	Remplacement par une cle de 44 caracteres
ContainerCreating bloque (postgres, meilisearch)	Multi-Attach : les PVC iscsi-retain sont RWO, l'ancien pod occupe encore le volume	Suppression manuelle des anciens pods pour liberer les volumes
Pipeline CI echoue (no Dockerfile)	Le template CI attend un Dockerfile a la racine	Creation d'un Dockerfile racine pour le backend

7. Tests et Validation

7.1. Verification des pods et services

Une fois l'ensemble des incidents resolus, la commande `kubectl -n annuaire get pods` confirme que tous les pods sont operationnels :

NAME	READY	STATUS	AGE
annuaire-backend-67fdd974b9-x9xp8	1/1	Running	...
annuaire-frontend-8464b7b487-8fqw7	1/1	Running	...

```
annuaire-meilisearch-65fcc5fbbd-7k6ql 1/1 Running ...
annuaire-postgres-5b66d6d7dd-6nq5f 1/1 Running ...
```

La commande `kubectl -n annuaire get deploy` confirme que les 4 deployments sont en état `1/1 READY` :

NAME	READY	UP-TO-DATE	AVAILABLE
annuaire-backend	1/1	1	1
annuaire-frontend	1/1	1	1
annuaire-meilisearch	1/1	1	1
annuaire-postgres	1/1	1	1

7.2. Test d'accès à l'application

L'application est accessible à l'adresse `https://annuaire.pharmacie.u-pariscite.fr`. Le certificat TLS Let's Encrypt est valide et généré automatiquement par `cert-manager`. L'endpoint de santé du backend répond correctement :

```
{"status": "healthy", "service": "annuaire-backend"}
```

Le pipeline CI/CD GitLab est fonctionnel (pipeline #797 passe avec succès). La création d'un tag Git déclenche automatiquement la construction de l'image et le déploiement via Flux.

7.3. Création des comptes utilisateurs

Les comptes administrateurs ont été créés directement sur la base de données de production via `kubectl exec` :

```
kubectl -n annuaire exec deploy/annuaire-backend -- python -c '...'
```

Deux comptes administrateurs ont été créés avec des liens d'invitation permettant de définir un mot de passe :

- **Mohamed ABIDI** (admin) : `mohamed.abidi@u-pariscite.fr`
- **Laurent FELICITE** (admin) : `laurent.felicite@u-pariscite.fr`

Le compte administrateur par défaut (`admin@u-paris.fr`) créé automatiquement par le script de seed a été supprimé après la création des comptes nominatifs.

8. Conclusion

Le déploiement de l'application Annuaire du Personnel sur le cluster Kubernetes de la faculté est pleinement opérationnel. Les objectifs fixés sont atteints :

- L'application est déployée en production avec quatre services (frontend, backend, PostgreSQL, MeiliSearch) fonctionnant dans un namespace dédié.
- L'ensemble des variables d'environnement est configuré (base de données, LDAP, SMTP, CAS, CORS) via des ConfigMaps et Secrets Kubernetes.
- Le pipeline CI/CD est en place et suit le même modèle que les autres applications de la faculté : un simple `git tag` suivi d'un `git push` suffit pour déployer une nouvelle version.
- L'application est accessible en HTTPS sur `annuaire.pharmacie.u-pariscite.fr` avec un certificat Let's Encrypt valide.
- Les comptes administrateurs sont créés et le compte par défaut a été supprimé.

Les principaux incidents rencontrés (chemins d'images, secrets insuffisants, conflits de volumes, compatibilité CI) ont été résolus méthodiquement en s'appuyant sur les logs Kubernetes (`kubectl logs` , `kubectl describe pod`) et en analysant les configurations des applications existantes comme références.

Ce déploiement s'inscrit dans la démarche d'hébergement centralisé des applications internes de la Faculté de Pharmacie sur une infrastructure Kubernetes gérée en GitOps, garantissant reproductibilité, traçabilité et automatisation des mises en production.